Optimisation des échantillonneurs exacts en précision arbitraire

Versions `_opt` des cinq algorithmes du pipeline

Dépôt `FLDR-experiments`

19 octobre 2026

Résumé

Nous proposons une version optimisée de chacun des cinq échantillonneurs comparés dans la figure 4 de l'article (Integer-Alias, FLDR, ALDR, Rust-Alias et Fraction-Alias, tous en précision arbitraire), ainsi qu'un nouveau producteur de bits aléatoires. Chaque version garde la philosophie de son algorithme (même table ou même arbre, mêmes opérations exactes, fractions conservées pour Fraction-Alias, uniforme sur $[0, W)$ pour Rust-Alias) et consomme exactement le même nombre de bits aléatoires. Dans les conditions du pipeline, le tirage est de **20 à 61 fois plus rapide** selon l'algorithme ; la mémoire baisse de 28 à 78 % pour quatre algorithmes sur cinq. Ce travail a aussi mis au jour **un bug dans le FLDR GMP d'origine**, qui échantillonnait une distribution biaisée ; il a été corrigé et remesuré. Les autres algorithmes d'origine n'ont pas été modifiés. Enfin, une variante `alias.integers_v2` remplace l'uniforme et le Bernoulli de Lumbroso par un seul mot de 64 bits par tirage : environ 2 fois plus rapide qu'`alias.integers_opt`, pour 64 bits par tirage au lieu de 17,6 (section 10).

1 Résumé des résultats

Algorithme	Temps médian / tirage		Gain	Mémoire	Bits / tirage
	original	optimisé	(min–max)	(médiane)	(identiques)
Integer-Alias	594 ns	13,3 ns	×45 (32–51)	−28 %	17,59
FLDR*	664 ns	33,6 ns	×20 (15–29)	−78 %	10,66
ALDR	599 ns	16,6 ns	×37 (21–50)	−78 %	8,29
Rust-Alias	1918 ns	62,6 ns	×30 (26–36)	+9 %	185,20
Fraction-Alias	747 ns	12,4 ns	×61 (38–69)	−30 %	17,59

TABLE 1 – Résultats sur les 201 distributions du protocole de la figure 4 ($n = 20\,484$, $w \approx 4{,}7 \cdot 10^{50}$, 10^7 tirages, 14 exécutions parallèles). Les gains sont calculés distribution par distribution. *FLDR original après correction du bug (section 5).

Ces gains viennent de deux sources, qu'on peut séparer sur Integer-Alias (distribution `d.00100`, exécution seule, 10^7 tirages) : l'algorithme optimisé avec l'ancien producteur de bits passe de 3,83 s à 0,56 s (×6,8), puis le nouveau producteur le ramène à 0,10 s (×5,4), soit ×37 au total.

2 Pourquoi le pipeline ralentissait les mesures

Le pipeline lance 14 mesures en parallèle. Nous avons mesuré la même distribution en faisant varier le nombre de mesures simultanées (temps `clock()` pour 10^7 tirages, avec l'ancien producteur de bits) :

Mesures en parallèle	Integer-Alias optimisé	Integer-Alias original
1	0,56 s	3,83 s
5	0,92 s	5,44 s
10	1,65 s	6,12 s
14	1,96 s	8,10 s

Trois causes s'additionnent.

1. **Surcharge du processeur.** Chaque mesure a deux threads : le tirage et le producteur de bits. Avec 14 mesures, cela fait 28 threads actifs pour 14 cœurs, dont 4 cœurs « efficacité » nettement plus lents que les 10 cœurs « performance » de l'Apple M4.
2. **L'ancien producteur ne s'arrête jamais.** Quand son buffer de 64 mots est plein, il boucle sur `sched_yield()` et occupe un cœur en permanence. Or `clock()` additionne le temps CPU de tous les threads du processus : ce temps perdu est compté dans le temps de tirage, et il augmente quand la machine est chargée.
3. **Ressources partagées.** Le ralentissement apparaît dès 5 mesures, alors qu'il reste des cœurs performance libres : les mesures se disputent aussi le cache et la mémoire. Chaque mesure balaie par exemple 128 Mo (`flush_cache`) au début et à la fin, pendant que les autres mesurent. En charge, la fréquence des cœurs baisse également.

Le protocole reste identique pour tous les algorithmes, donc la comparaison reste équitable ; mais les temps absolus sont gonflés, d'autant plus que l'algorithme est rapide. Le nouveau producteur (section 3) supprime la deuxième cause pour les versions optimisées. La part exacte de la troisième cause n'a pas été isolée.

3 Le nouveau producteur de bits aléatoires

L'ancien producteur (`flip.c`) appelle `arc4random_buf` pour chaque mot de 32 bits. Chaque appel coûte environ 39 ns, alors qu'un appel sur un bloc de 4 Ko coûte 0,8 ns par mot de 32 bits :

Taille du bloc (mots de 64 bits)	Coût par mot de 32 bits
1	14,3 ns
8	9,9 ns
64	1,8 ns
512	0,8 ns

Le nouveau producteur (`c/flip_opt.c` et `c/flip_opt.h`) garde le même principe, un thread qui remplit en continu un buffer de bits cryptographiques (`arc4random`), avec les changements suivants :

— un buffer circulaire de 256 Ko en mots de 64 bits, rempli par blocs de 512 mots ;
— `head` et `tail` sur des lignes de cache distinctes, publiés avec une sémantique *acquire/release* ; le consommateur ne relit `tail` que lorsqu'il a épuisé les mots qu'il sait disponibles ;
— quand le buffer est plein, le producteur dort 50 µs au lieu de boucler. Le buffer contient environ deux millions de bits, soit plusieurs millisecondes de tirages ;
— le consommateur lit les bits directement dans le mot courant (fonctions *inline*), avec `ctz` pour trouver la première pièce égale à 1.

Les bits sont lus du poids faible vers le poids fort, comme dans `flip.c`. Le compteur `NUM_RNG_CALLS` est incrémenté de 2 par mot de 64 bits : la colonne « bits » du pipeline garde donc la même unité.

`main.c` choisit le producteur d'après le nom de l'échantillonneur : les échantillonneurs dont le nom se termine par `_opt` utilisent le nouveau, les autres gardent `flip.c`, qui n'a pas été modifié.

4 Optimisations par algorithme

Les cinq versions optimisées partagent deux briques, définies dans `flip_opt.h`.

Uniforme sur $[0, n)$. Même Fast Dice Roller de Lumbroso que `uniform` et `uniform_with_gmp`, sur des entiers machine (l'indice d'une cellule tient sur 32 bits), avec le nombre de bits du premier bloc précalculé.

Bernoulli(t/d) à chiffres précalculés. Le Bernoulli de Lumbroso tire des pièces $c_1, c_2, \ldots$ et renvoie le chiffre binaire d_k de t/d à la première pièce $c_k = 1$; si le développement se termine exactement au chiffre k, il renvoie directement c_k. On a donc $\Pr(1) = \sum_k d_k 2^{-k} = t/d$. Au prétraitement, on calcule une fois pour toutes

$$D = \left\lfloor t \cdot 2^{31}/d \right\rfloor = (d_1 \ldots d_{31})_2, \qquad \rho = t \cdot 2^{31} \bmod d,$$

et on stocke les 31 chiffres, plus un drapeau qui vaut 1 si $\rho = 0$. Au tirage, on cherche la première pièce égale à 1 avec `ctz`, puis :

- si le développement est fini et se termine au chiffre L, on lit au plus L pièces. Si la première pièce à 1 est en position $k \le L$, on renvoie d_k ; pour $k = L$ c'est bien $c_L = 1$. Si les L pièces sont nulles, on renvoie $0 = c_L$;
- sinon, si la première pièce à 1 est en position $k \le 31$, on renvoie d_k ;
- si les 31 premières pièces sont nulles (probabilité 2^{-31}), on reprend la boucle GMP d'origine à partir de l'état ρ. Le résultat reste exact.

Les pièces sont lues dans le même ordre et en même nombre que dans l'original.

4.1 Integer-Alias (`alias.integers_opt`)

- Même table que l'Algorithme 1 de l'article, construite dans le même ordre (piles L et S, LIFO, objet virtuel), et même rejet de l'objet virtuel.
- Uniforme et Bernoulli($T[i]/cs$) selon les deux briques ci-dessus : plus aucune opération GMP au tirage, sauf dans le repli de probabilité 2^{-31}.
- Cellules de 16 octets (chiffres, objet, alias, position du seuil). Les seuils exacts sont stockés en octets, uniquement pour les cellules avec alias, rangées en tête de table ; l'ordre des cellules est indifférent puisqu'elles sont tirées uniformément.

4.2 FLDR (`rej.enc_opt`)

- Même arbre DDG que `rej.enc_gmp`, lu dans le même fichier (encodage `pack_tree` construit en Python à partir des poids en précision arbitraire), parcouru comme dans le FLDR de Saad (`sample_rejection_encoding`).
- L'encodage ne contient que des positions mémoire et des étiquettes : il est stocké en `int32` au lieu de `mpz_t` (les valeurs sont vérifiées à la lecture).
- Le parcours avance bit par bit dans le mot courant, sans appel de fonction par bit.

4.3 ALDR (`aldr_opt`)

- Même arbre aplati que `aldr_gmp` (profondeur $K = 2k$, facteur $c = \lfloor 2^K/m \rfloor$). La construction d'origine recalcule le produit GMP $c \cdot a_i$ pour chacun des $K + 1$ niveaux, soit environ $336 \times 20\,484$ multiplications ; la version optimisée le calcule une fois par objet, dans le même ordre. Les arbres obtenus sont identiques (vérifié sur 3 000 distributions).
- Les largeurs de niveau et les feuilles sont stockées en `uint32`. Les compteurs du parcours tiennent en entiers machine, car un arbre DDG a au plus $n + 1$ nœuds internes par niveau.
- Parcours bit par bit dans le mot courant.

4.4 Rust-Alias (`alias.rust_opt`)

- Même table que `alias.rust_gmp` (construite par `weighted_alias_new_gmp` : poids multipliés par n, cellules de capacité W).
- Même tirage, fidèle à la crate Rust : une cellule uniforme, puis un entier *uniforme sur* $[0, W)$ *en précision arbitraire* (environ 185 bits), comparé au seuil. Nous avons volontairement gardé cet uniforme complet, qui fait le coût en bits de cette méthode.
- L'uniforme sur $[0, W)$ suit le même Fast Dice Roller, mais sur des entiers GMP préalloués (aucune allocation par tirage), avec les limbs aléatoires écrits directement. Les seuils sont stockés en limbs contigus de largeur fixe.
- La mémoire augmente de 9 %. Les seuils sont stockés à largeur fixe (celle de W), alors que la mesure d'origine ne compte que les octets utiles de chaque entier ; s'y ajoutent les deux entiers de travail préalloués.

4.5 Fraction-Alias (`alias.fractions_opt`)

- Même table que `alias.fractions_gmp` (construction de Vose sur des probabilités rationnelles exactes). **Les fractions sont conservées** : chaque cellule garde son numérateur et son dénominateur exacts, et le Bernoulli(a/b) utilise les 31 premiers chiffres binaires de la fraction, avec repli exact sur la fraction elle-même.
- Cellules de 16 octets ; numérateurs et dénominateurs en octets, uniquement pour les cellules avec alias.

5 Le bug du FLDR GMP d'origine

L'encodage FLDR (`pack_tree`, Saad et al.) range une feuille dans *une* case (`[-label]`) et un nœud interne dans deux cases (positions de ses enfants). Le parcours correct, celui de `sample_rejection_encoding`, est :

```
c = enc[c + b];          // position de l'enfant
if (enc[c] < 0) ...      // feuille : etiquette -enc[c]
```

La version GMP d'origine (`sample_ky_encoding_gmp`) testait `enc[c + b] < 0`. Arrivée sur une feuille, elle lisait donc un bit de plus : avec ce bit à 0 elle trouvait bien la feuille, mais avec ce bit à 1 elle lisait la case suivante, qui appartient à un autre nœud. Le parcours repartait alors d'un mauvais endroit.

Distribution	Version	p-valeur χ^2	Pire objet (obs. / attendu)	Bits / tirage
`d.00100`	GMP d'origine	0	31 732 / 52 066	12,97
`d.00100`	GMP corrigée	0,23	conforme	10,67
`d.00180`	GMP d'origine	0	804 / 1 634	18,64
`d.00180`	GMP corrigée	0,45	conforme	16,50

TABLE 2 – $2 \cdot 10^6$ tirages par cas, comparés aux probabilités exactes w_i/w.

Conséquences sur les résultats de l'article :

- la distribution échantillonnée par « FLDR (GMP) » n'était pas la bonne ;
- sa consommation de bits était surestimée d'environ 2,2 bits par tirage (12,88 en moyenne au lieu de 10,66) ;
- son temps était *sous-estimé* : 492 ns en médiane dans la série biaisée, 664 ns après correction.

La correction change trois lignes de `c/sample.c`. Elle a été validée de deux façons : sur un flux de bits identique, la version corrigée donne le même résultat et consomme le même nombre de bits que le FLDR de Saad (250 000 tirages sur 5 fichiers, 0 divergence) ; et elle passe le test du χ^2. `rej.enc_gmp` a ensuite été remesuré dans les conditions du pipeline.

6 Validation

6.1 Équivalence chemin par chemin

Un test remplit l'ancienne file de bits (mots de 32 bits) et la nouvelle (mots de 64 bits) avec exactement la même suite de bits, exécute l'original puis la version optimisée, et compare le résultat et le nombre de bits consommés.

Composant comparé à l'original	Cas testés	Divergences
Bernoulli à chiffres précalculés vs `bernoulli_with_gmp`	397 847	0
dont repli GMP (31 chiffres / 3 chiffres)	119 020 / 254 975	0
Uniforme vs `uniform`	126 669	0
ALDR : arbre et tirages vs `aldr_gmp` (3 000 arbres)	60 000	0
FLDR optimisé vs FLDR de Saad (4 fichiers)	80 000	0
FLDR GMP corrigé vs FLDR de Saad (5 fichiers)	250 000	0

Les seuils testés vont de 1 à 400 bits, dont un quart avec un développement binaire fini ; les flux de bits sont uniformes, peu denses ou très creux, pour provoquer de longues suites de zéros. La version « 3 chiffres » force le repli GMP. Pour l'uniforme, la comparaison chemin par chemin n'a de sens que si le premier bloc de bits ne franchit pas une frontière de mot de 32 bits : au-delà, l'ancienne et la nouvelle file assemblent les mêmes bits dans un ordre différent, et les deux résultats restent uniformes.

Pour Integer-Alias, Rust-Alias et Fraction-Alias, l'ordre des cellules ou celui des limbs aléatoires diffère de l'original, et la comparaison chemin par chemin ne s'applique pas directement. Leur exactitude repose sur les tests ci-dessus pour leurs briques communes, et sur les tests statistiques qui suivent.

6.2 Tests statistiques

Les dix échantillonneurs (cinq originaux, cinq optimisés) ont été testés par un χ^2 contre les probabilités exactes, sur trois distributions réelles du protocole (entropies basse, moyenne et haute), avec $2 \cdot 10^6$ tirages chacun et le vrai producteur de bits. Toutes les p-valeurs sont compatibles avec une distribution exacte (de 0,06 à 0,98) ; aucun tirage hors support. La consommation de bits de chaque version optimisée coïncide avec celle de son original.

Pour Integer-Alias, l'uniformité des p-valeurs a aussi été vérifiée sur 40 répétitions (test de Kolmogorov–Smirnov : $p = 0{,}28$ pour la version optimisée, 0,88 pour l'original).

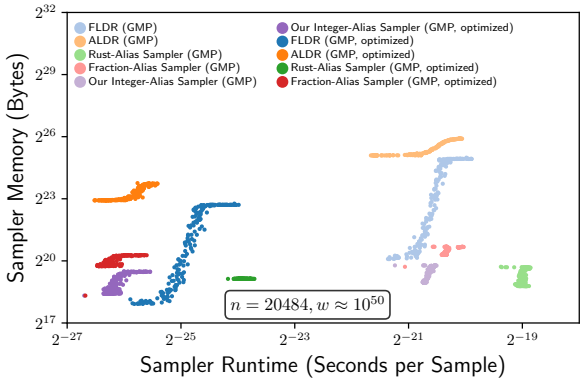
7 Mesures expérimentales

Protocole de la figure 4 : 201 distributions de `experiments/distributions_entropy`, 10^7 tirages par distribution, 14 mesures en parallèle, deux passes, MacBook Pro Apple M4 (14 cœurs). Les dix échantillonneurs ont été remesurés dans la même session (30 minutes), puis `rej.enc_gmp` une fois corrigé.

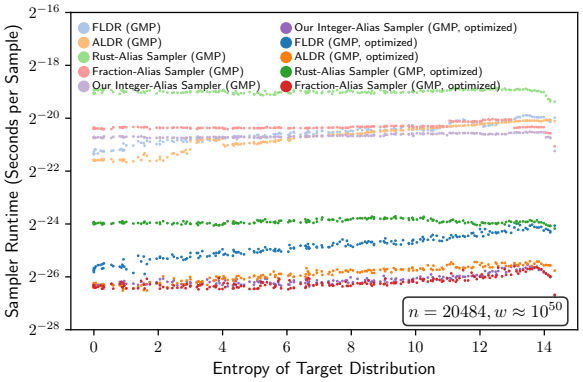
7.1 Temps

Toutes les versions optimisées sont plus rapides que tous les originaux, sur toute la plage d'entropie (figure 1b). Parmi elles (figure 2), Integer-Alias, Fraction-Alias et ALDR sont les plus rapides (10 à 22 ns par tirage), suivis de FLDR (16 à 60 ns) puis de Rust-Alias (53 à 73 ns), qui paie son uniforme de 185 bits. Les méthodes Alias gardent un temps quasi constant ; FLDR et ALDR croissent avec l'entropie, comme leurs originaux.

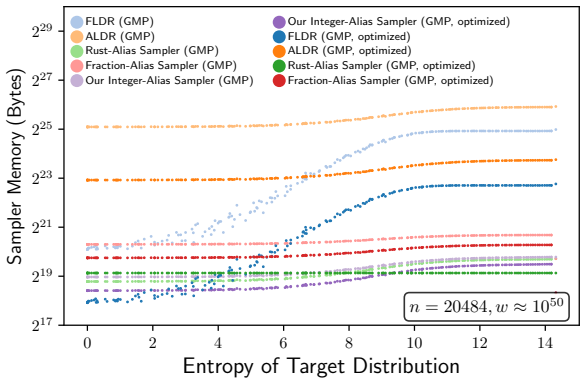
Exécutés seuls (sans la charge du pipeline), les gains sont du même ordre :



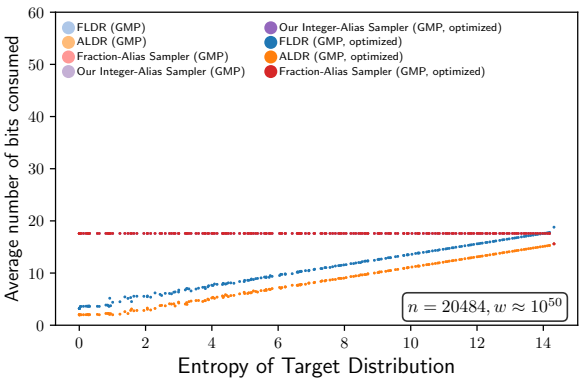
(a) Temps et mémoire.



(b) Temps en fonction de l'entropie.

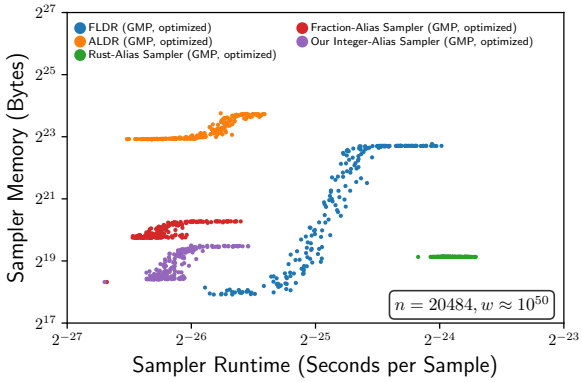


(c) Mémoire en fonction de l'entropie.

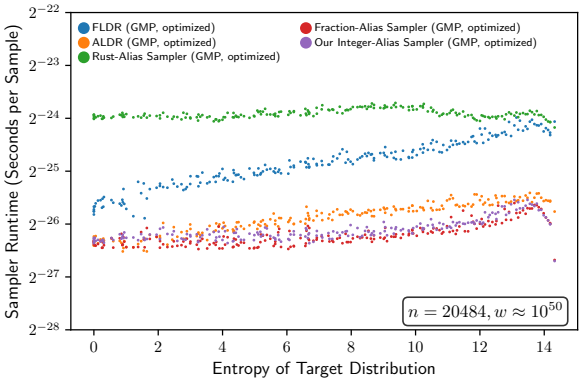


(d) Bits consommés en fonction de l'entropie.

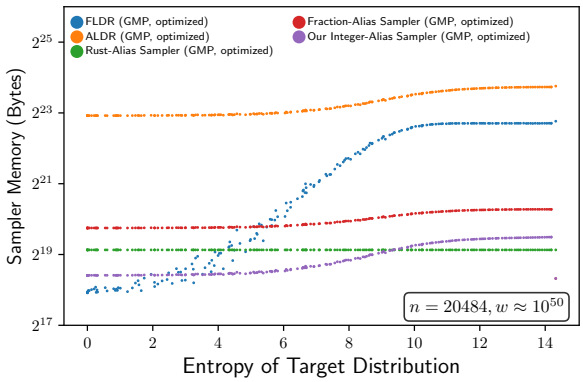
FIGURE 1 – Figures du notebook `figure-4-interpolation_Antoine.ipynb`. Chaque algorithme a sa teinte : clair pour l'original, foncé pour la version optimisée. Sur la figure (d), les points des versions optimisées recouvrent ceux des originaux (consommation identique) ; Rust-Alias (185 bits) est hors échelle.



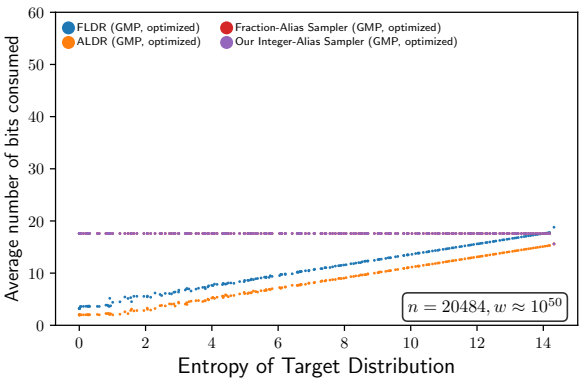
(a) Temps et mémoire.



(b) Temps en fonction de l'entropie.



(c) Mémoire en fonction de l'entropie.



(d) Bits consommés en fonction de l'entropie.

FIGURE 2 – Mêmes mesures que la figure 1, avec les seules versions optimisées et des axes resserrés, pour les comparer entre elles. Les couleurs sont celles de la figure 1. Sur la figure (d), Rust-Alias (185 bits) est hors échelle.

10^7 tirages, exécution seule	`d.00000`	`d.00100`	`d.00180`
Integer-Alias	3,79 s → 0,10 s	4,09 s → 0,10 s	4,25 s → 0,14 s
FLDR (corrigé)	3,20 s → 0,16 s	4,02 s → 0,30 s	5,98 s → 0,45 s
ALDR	3,38 s → 0,11 s	6,10 s → 0,15 s	8,35 s → 0,18 s
Rust-Alias	8,64 s → 0,48 s	8,58 s → 0,51 s	8,72 s → 0,51 s
Fraction-Alias	4,62 s → 0,10 s	5,01 s → 0,10 s	5,10 s → 0,14 s

Variabilité entre sessions. Les temps des originaux mesurés ce jour diffèrent de ceux de la série précédente de −21 à +8 % (par exemple Integer-Alias : 616 ns puis 594 ns ; Rust-Alias : 1774 ns puis 1918 ns). C'est l'ordre de grandeur de la variabilité due aux conditions de la machine ; elle ne change pas les conclusions.

7.2 Mémoire

FLDR et ALDR passent d'entiers `mpz_t` (16 octets d'en-tête plus les limbs) à des entiers 32 bits : −78 % chacun. Integer-Alias (−28 %) et Fraction-Alias (−30 %) gagnent grâce aux seuils stockés en octets, uniquement pour les cellules avec alias. Rust-Alias augmente de 9 % (voir plus haut). Integer-Alias optimisé reste l'échantillonneur le plus compact de la comparaison.

7.3 Bits aléatoires

La consommation de chaque version optimisée est égale à celle de son original (Integer-Alias 17,59 ; FLDR corrigé 10,66 ; ALDR 8,29 ; Rust-Alias 185,20 ; Fraction-Alias 17,59 bits par tirage). Les analyses de l'article (proposition 4, lemme 3) s'appliquent donc sans changement, à ceci près que les valeurs de FLDR doivent être reprises après correction.

8 Mesures séquentielles des versions optimisées

Pour supprimer la contention décrite en section 2, les cinq versions optimisées ont été remesurées avec la version séquentielle du pipeline (`experiments/pipeline_seq.sh`), avec en plus `alias.integers_v2` (section 10). Le protocole est celui de la figure 4 : 201 distributions de `distributions_entropy`, 10^7 tirages par distribution, temps `clock()`. Mais une seule mesure tourne à la fois, chaque échantillonneur est mesuré une fois, et les six échantillonneurs sont mesurés à la suite pour chaque distribution. L'exécution complète prend environ 9 minutes sur le M4 Pro. Les figures sont produites par `figure-4-interpolation_Antoine.ipynb`, restreint à ces six échantillonneurs (fichiers `figure-4-interpolation-opt-*`).

Temps par tirage (ns)	Pipeline parallèle (médiane)	Séquentiel (médiane)	Séquentiel (min–max)
Integer-Alias	13,3	10,5	8,5–17,3
FLDR	33,6	28,3	14,4–51,4
ALDR	16,6	14,1	9,2–20,3
Rust-Alias	62,6	50,4	44,5–57,0
Fraction-Alias	12,4	10,1	8,1–16,6
Integer-Alias v2	–	4,4	4,1–7,1

TABLE 3 – Versions optimisées et v2, 201 distributions, série du 22 octobre. La colonne « pipeline parallèle » reprend la table 1 ; v2 n'y a pas été mesuré.

Sans contention, les temps baissent de 15 à 21 % (en médiane), et le classement des cinq versions optimisées ne change pas. Integer-Alias et Fraction-Alias sont les plus rapides et presque constants en fonction de l'entropie ; viennent ensuite ALDR, puis FLDR, qui croît nettement avec l'entropie, puis Rust-Alias. Integer-Alias v2 est le plus rapide de tous, sur toute la plage d'entropie : 4,4 ns en

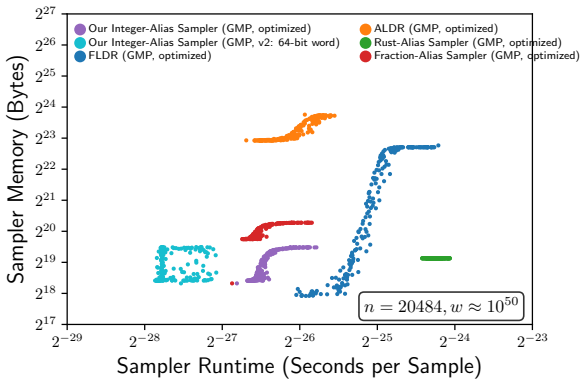
médiane, 2,3 fois moins qu'Integer-Alias (section 10). La mémoire et les bits ne dépendent pas du mode d'exécution : ils sont identiques à ceux de la section précédente. Les originaux (`_gmp`) n'ont pas été remesurés en séquentiel. Les gains de la table 1 restent donc ceux du pipeline parallèle, où les deux versions sont mesurées dans les mêmes conditions.

Pourquoi Integer-Alias dépasse Rust-Alias en mémoire vers $H \approx 10$. Sur la figure 3c, la courbe d'Integer-Alias passe au-dessus de celle de Rust-Alias vers une entropie de 10. La raison tient à la façon dont chacun stocke ses seuils.

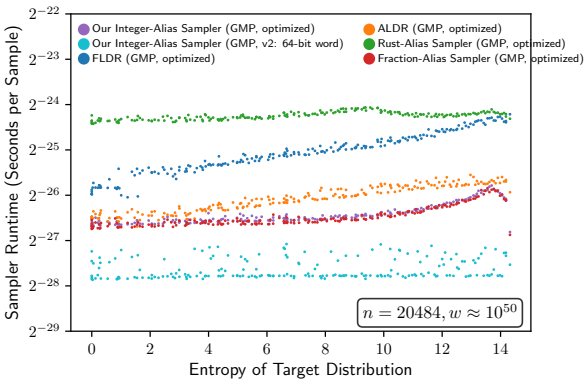
— **Rust-Alias** stocke chaque seuil à largeur fixe, celle de W : 3 limbs de 8 octets. Avec l'alias (4 octets), cela fait 28 octets par objet, soit $28 \times 20\,484 \approx 574$ Ko pour toutes les distributions. Sa courbe est plate.

— **Integer-Alias** stocke des cellules de 16 octets ($16 \times 20\,485 \approx 328$ Ko), plus chaque seuil exact T_i avec ses seuls octets utiles, et seulement pour les cellules avec alias. À basse entropie, les petits objets ont des poids minuscules (beaucoup valent 1) : chaque seuil tient sur environ 1 octet, et le total reste vers 350 Ko. Quand l'entropie monte, les poids des petits objets se rapprochent de $cs \approx 2 \cdot 10^{46}$, soit environ 20 octets par seuil : le total atteint 720 Ko.

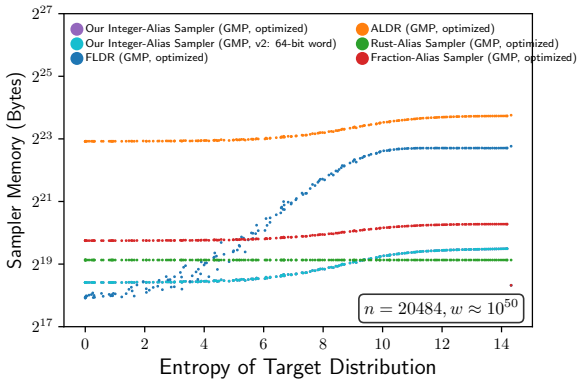
Les deux courbes se croisent quand les seuils d'Integer-Alias dépassent en moyenne environ $(574 - 328)/20\,485 \approx 12$ octets, ce qui arrive vers $H \approx 10$. Le dernier point, la distribution uniforme, retombe à 328 Ko : aucune cellule n'a d'alias, donc il n'y a aucun seuil à stocker.



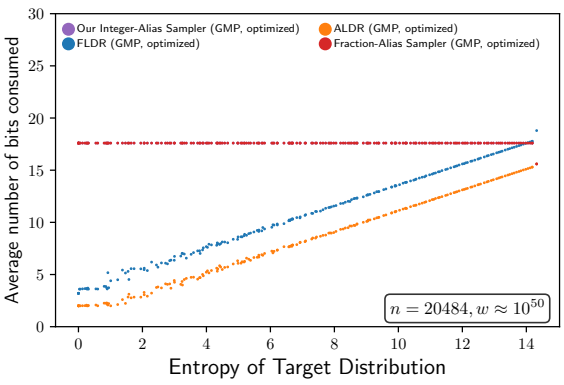
(a) Temps et mémoire.



(b) Temps en fonction de l'entropie.



(c) Mémoire en fonction de l'entropie.



(d) Bits consommés en fonction de l'entropie.

FIGURE 3 – Versions optimisées et Integer-Alias v2 (en cyan), pipeline séquentiel (`pipeline_seq.sh`). Sur la figure (c), v2 a la même mémoire qu'Integer-Alias et leurs points se recouvrent. Sur la figure (d), Integer-Alias et Fraction-Alias consomment le même nombre de bits et leurs points se recouvrent ; Rust-Alias (185 bits) et v2 (64 bits) sont hors échelle (voir la figure 4).

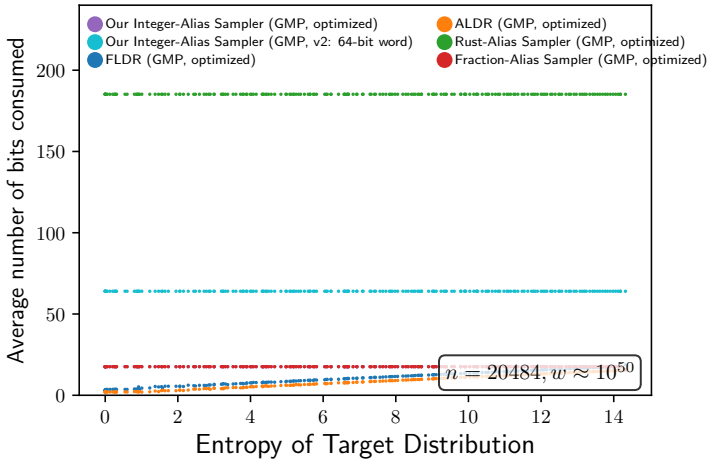


FIGURE 4 – Bits consommés, échelle complète : Rust-Alias (185 bits), Integer-Alias v2 (64 bits), puis les autres versions optimisées, entre 2 et 19 bits.

8.1 Temps de preprocessing

Mesure. Le programme `c/measure_preprocess.c` lit d'abord les poids, sans chronométrer cette lecture, puis mesure la seule construction de la structure de tirage à partir des poids. Il la répète au moins 5 fois et pendant au moins 50 ms, et garde le meilleur temps (horloge `CLOCK_MONOTONIC_RAW`). La construction est celle qu'utilise chaque échantillonneur : `preprocess_alias_integers_opt`, `preprocess_alias_integers_v2` (la même, plus le calcul des 32 chiffres D_i), `preprocess_aldr_opt`, et pour Rust-Alias et Fraction-Alias la construction d'origine suivie de la conversion vers la table optimisée. La commande `./pipeline_seq.sh distributions_entropy measure-preprocess` (aussi lancée par `run-all`) écrit un fichier `<sampler>.preprocess` par échantillonneur ; elle prend environ 2 minutes.

FLDR porté en C. L'arbre DDG de FLDR était construit en Python (`src/rejection.py`, `matrix.py`, `tree.py`, `packing.py`), et le C ne faisait que relire l'encodage. Pour comparer des temps C entre eux, nous avons écrit `preprocess_fldr_opt` (`c/fldr_opt.c`). Comme la matrice de FLDR est dyadique ($\sum_i M_i = 2^k$), l'arbre n'a pas d'arête de retour, et il se décrit profondeur par profondeur. À la profondeur d, les enfants des nœuds internes de la profondeur $d-1$ sont, de gauche à droite, les nœuds internes, puis les feuilles des lignes dont le bit $k-d$ vaut 1, par ligne décroissante. L'encodage est ensuite écrit en préordre, comme `pack_tree`. On évite ainsi les indices de nœuds de $k \approx 170$ bits de la version Python. Sur les 201 distributions, l'encodage produit est *identique* à celui des fichiers `.rej.enc_opt` écrits par Python (`measure_preprocess.out.opt check-fldr`).

Une particularité des données. Pour la distribution uniforme (`d.00200`), la somme des poids dépasse de $1{,}25 \cdot 10^{34}$ (soit $2{,}7 \cdot 10^{-17}$ en relatif) le Z écrit en première ligne du fichier, à cause de l'arrondi de `normalize_to_Z`. Les méthodes Alias et ALDR normalisent par la somme des poids, et tirent bien l'uniforme. FLDR utilise les probabilités w_i/Z : il tire aussi l'uniforme, grâce au rejet, mais avec $k = 169$ au lieu de 15 et un arbre plus grand. Le portage en C reprend cette convention, pour rester identique à Python.

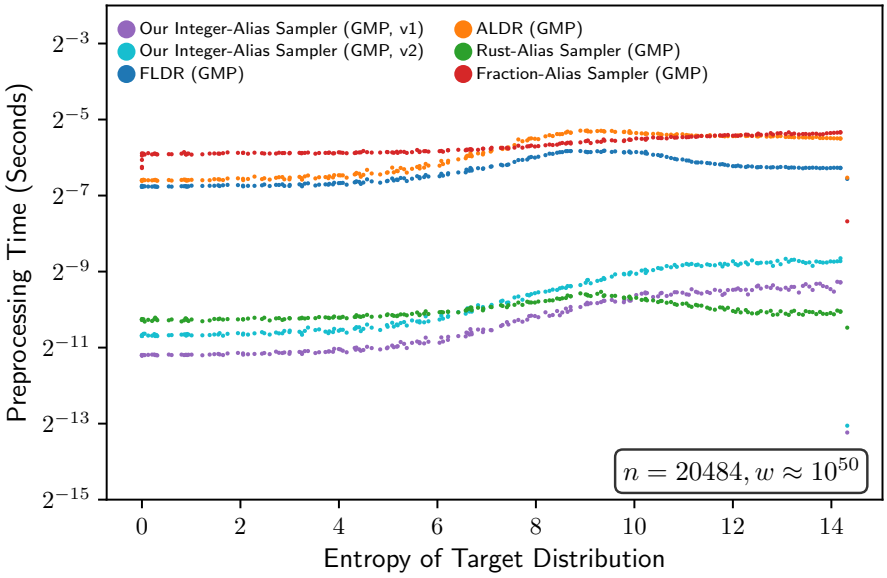


FIGURE 5 – Temps de construction de la structure à partir des poids, en fonction de l'entropie (201 distributions, $n = 20\,484$). Integer-Alias v1 est `alias.integers_opt`.

Construction (ms)	Médiane	Min	Max
Integer-Alias (`opt`)	0,68	0,10	1,62
Integer-Alias v2	1,03	0,12	2,49
Rust-Alias	0,93	0,70	1,35
FLDR	12,85	9,15	17,72
ALDR	17,04	10,22	25,58
Fraction-Alias	18,34	4,88	24,84

— **Integer-Alias est la construction la plus rapide** : de 0,1 à 1,6 ms, soit en médiane 19 à 27 fois moins que FLDR, ALDR et Fraction-Alias. Sur les mêmes distributions, la construction d'origine (`preprocess_gmp_alias_integers`) prend environ 1 ms. La version optimisée est donc du même ordre : elle ajoute une division GMP par cellule pour les 31 chiffres du seuil, mais range les seuils en octets.

— **Elle croît avec l'entropie**, entre $H \approx 6$ et $H \approx 10$, pour la même raison que sa mémoire (section 8) : les seuils T_i des petits objets passent d'environ 1 octet à environ 20, et la division GMP qui calcule leurs chiffres coûte plus cher. La distribution uniforme n'a aucune cellule avec alias : 0,1 ms.

— **v2 coûte environ 50 % de plus qu'`opt`** : il construit la table d'`opt`, puis fait une seconde division GMP par cellule pour les 32 chiffres D_i. On pourrait calculer D_i directement, avec une seule division ; nous ne l'avons pas fait, car le preprocessing reste sous 2,5 ms.

— **Rust-Alias** est presque constant (environ 1 ms) : sa construction ne multiplie les poids que par n, sans division par cellule.

— **FLDR et ALDR** parcourent les bits de chaque ligne à chaque profondeur : $k \approx 170$ niveaux pour FLDR et $K = 2k \approx 340$ pour ALDR, sur $n \approx 20\,000$ lignes. Leur coût suit le nombre de feuilles, qui croît avec l'entropie.

— **Fraction-Alias** manipule des rationnels GMP (`mpq_t`), qui sont normalisés par un calcul de pgcd après chaque opération : c'est la construction la plus lente.

9 Peut-on faire mieux pour l'uniforme et le Bernoulli ?

Un tirage d'Integer-Alias se compose d'une uniforme sur les q cellules (Fast Dice Roller, FDR) et d'un Bernoulli(T_i/cs) (algorithme de Lumbroso, ici avec chiffres précalculés). Ces deux briques sont-elles optimales en nombre de bits, et le sont-elles aussi en temps ? Les mesures de cette section sont faites avec `rapport_optimisation/bench_temps/bench.c` ; les calculs exacts avec `theory.py`, les tests avec `chi2.py`, dans le même dossier.

9.1 En bits : optimales pour un tirage isolé

Le théorème de Knuth et Yao (1976) dit que tout algorithme qui tire un objet de loi p à partir de bits frais en consomme en moyenne au moins $H(p)$, et que l'arbre de Knuth–Yao (KY) atteint le minimum, compris entre $H(p)$ et $H(p) + 2$.

Uniforme. Nous avons calculé exactement le nombre moyen de bits du FDR (en suivant la chaîne des états de la boucle) et celui de l'arbre KY de l'uniforme sur $[0, n)$ (à partir des chiffres binaires de $1/n$) :

n	$\log_2 n$	FDR	KY
3	1,585	2,667	2,667
7	2,807	3,429	3,429
1 000	9,966	10,151	10,151
16 385	14,000	15,999	15,999
20 485	14,322	15,597	15,597

Les deux coïncident pour toutes les valeurs testées (11 valeurs de n, de 3 à 24 576) : aucun algorithme qui tire une uniforme isolée ne consomme moins de bits que le FDR. Sur la table du protocole ($q = 20\,485$), il perd 1,27 bit par rapport à $\log_2 q$, et cette perte est inévitable pour un tirage isolé.

Bernoulli. Si p n'est pas dyadique, les développements binaires de p et de $1 - p$ sont complémentaires : l'arbre KY de Bernoulli(p) a exactement une feuille à chaque profondeur, et coûte $\sum_k k\, 2^{-k} = 2$ bits. Le Bernoulli de Lumbroso est précisément cet arbre (pièce k égale à 1 : feuille de profondeur k, étiquette d_k). Si p est dyadique, les deux s'arrêtent au dernier chiffre. Le Bernoulli est donc optimal pour un tirage isolé.

Mais pas pour une suite de tirages. Ces bornes portent sur *un* tirage qui part de bits frais. Quand on tire une suite d'objets, les bits « inutilisés » par un tirage peuvent servir au suivant, et la borne devient H par tirage. Pour la structure d'Integer-Alias, le minimum est alors l'entropie du couple (cellule, résultat du Bernoulli), $\log_2 q + \mathbb{E}_i[h(T_i/cs)]$, où h est l'entropie binaire. Sur les distributions du protocole, les seuils sont presque tous très proches de 0 (petits objets de poids $\ll cs$) : $\mathbb{E}[h] = 0{,}022$ bit sur `d.00100`, alors que le Bernoulli en dépense 2. Integer-Alias consomme 17,60 bits là où 14,34 suffisent pour la même table.

9.2 En temps : pas optimales

Il n'existe pas de borne inférieure théorique utile en temps : dans le modèle RAM avec des mots de 64 bits, un tirage coûte $\Omega(1)$ opérations, plus la lecture d'environ $H/64$ mots aléatoires. La question porte donc sur les constantes, et nous l'avons traitée par la mesure.

Variantes comparées. Toutes utilisent la même table (celle d'`alias.integers_opt`) et le même flux de bits (`flip_opt`).

— **Lemire** (2019) : uniforme par multiplication d'un mot de 64 bits par q, rejet de probabilité $< q/2^{64}$. Exacte, mais 64 bits par tirage.
— **Recyclage** : on garde d'un tirage à l'autre un état (U, M), avec U uniforme sur $[0, M)$; quand $M < 2^{32}$, on ajoute 32 bits frais ($U \leftarrow 2^{32}U + B$, $M \leftarrow 2^{32}M$). C'est le principe du codage arithmétique et de l'algorithme d'intervalle de Han et Hoshi (1997).
  — Uniforme sur $[0, q)$: on écrit $M = aq + r$. Si $U < aq$, on renvoie $U \bmod q$ et l'état devient $(\lfloor U/q \rfloor, a)$; sinon l'état devient $(U - aq, r)$ et on recommence. La seule information perdue est le choix entre ces deux branches.
  — Bernoulli(p), $p = T_i/cs$: on précalcule $P = \lfloor p\, 2^{64} \rfloor$ (8 octets par cellule) et $\ell = \lfloor PM/2^{64} \rfloor$ (une multiplication). Si $U < \ell$, on renvoie 1 et l'état devient (U, ℓ) ; si $U \geq \ell + 2$, on renvoie 0 et l'état devient $(U - \ell - 2, M - \ell - 2)$. Le cas restant (probabilité $\leq 2/M \leq 2^{-31}$) est tranché exactement en GMP avec des bits frais, puis l'état est abandonné.
— **cmp31** : Bernoulli par comparaison de 31 bits frais avec les 31 chiffres d'un coup (exact, 31 bits par appel).
— **Plancher inexact** : un mot de 64 bits, 32 bits pour la cellule et 32 bits comparés à un seuil arrondi. Ce n'est pas exact (biais de l'ordre de 2^{-32}), mais c'est à peu près ce que coûte au minimum une méthode Alias avec ce flux de bits.

Exactitude du recyclage. Par récurrence, sachant tout le passé, U est uniforme sur $[0, M)$. Le remplissage conserve cette propriété. Dans l'uniforme, conditionnellement à $U < aq$, le couple $(U \bmod q, \lfloor U/q \rfloor)$ est uniforme sur $[0, q) \times [0, a)$: le résultat est uniforme et indépendant du nouvel état. Dans le Bernoulli, soit $V = (U + \Theta)/M$ avec Θ uniforme sur $[0, 1)$ (jamais tirée) : V est uniforme sur $[0, 1)$, et le résultat vaut exactement $\mathbf{1}\{V < p\}$, car $U < \ell$ entraîne $V < p$ et $U \geq \ell + 2$ entraîne $V \geq p$. Dans chaque branche, le nouvel état est uniforme sur son intervalle. Les tirages successifs sont donc indépendants et de loi exacte. Vérification : χ^2 sur $5 \cdot 10^7$ tirages de `d.00100`, $p = 0{,}82$; test des paires successives (classes de masse $\approx 1/8$), $p = 0{,}80$. Les autres variantes exactes donnent des p-valeurs de 0,13 à 0,91.

Mesures. MacBook Pro M4 Pro, un seul processus, $2 \cdot 10^7$ tirages, meilleur de trois essais. On donne le temps réel par tirage, le temps CPU entre parenthèses (`clock()`, producteur compris, comme dans le pipeline) et les bits consommés.

Brique seule (`d.00100`)	Temps (ns)	(CPU)	Bits
Uniforme FDR (actuelle)	6,93	(7,54)	15,60
Uniforme Lemire	3,46	(5,21)	64,00
Uniforme recyclage	5,57	(6,11)	14,32
Bernoulli à chiffres, `ctz` (actuel)	2,88	(3,02)	2,00
Bernoulli recyclage	2,91	(2,99)	0,02
Bernoulli cmp31	3,90	(4,78)	31,00
Lecture de 18 bits du flux	2,76	(3,31)	18,00
Lecture d'une cellule au hasard	1,58	(1,63)	–

Tirage complet	Bits	`d.00000`	`d.00050`	`d.00100`	`d.00150`	`d.00200`
Entropie $H(p)$		0,00	3,73	7,08	10,74	14,32
Temps réel (ns par tirage)						
FDR + `ctz` (actuel)	17,6*	9,30	9,48	9,68	11,18	8,26
Lemire + `ctz`	66,0*	6,14	6,19	6,51	7,74	4,18
Recyclage + `ctz`	16,3*	5,81	6,02	6,50	7,83	4,97
Recyclage, 1 état	14,3–14,5	8,37	11,23	13,16	13,68	5,04
Recyclage, 2 états	14,3–14,5	**4,89**	**5,13**	**5,40**	**7,19**	**5,04**
Plancher inexact	64,0	4,14	4,20	4,32	4,39	4,47
Temps CPU (ns par tirage, producteur compris)						
FDR + `ctz` (actuel)		10,0	10,1	10,4	12,0	8,9
Recyclage, 2 états		5,4	5,7	6,0	7,8	5,6
Plancher inexact		5,9	6,0	6,1	6,2	6,3

TABLE 4 – `ctz` : Bernoulli à chiffres précalculés actuel. *Sur `d.00200`, 2 bits de moins (15,6 ; 64,0 ; 14,3) : c'est la distribution uniforme, $q = n$ et aucune cellule n'a d'alias, il n'y a donc pas de Bernoulli. Les écarts entre deux exécutions sont de l'ordre de 0,3 ns.

Lecture des résultats.

1. **L'uniforme FDR est le poste le plus coûteux** : environ 7 ns, deux fois Lemire. Après le premier bloc de $\lceil \log_2 q \rceil$ bits, la boucle lit un bit par itération, et sa sortie dépend du hasard : pour $q = 20\,485$, le premier bloc est accepté avec probabilité $20\,485/2^{15} \approx 0{,}63$ seulement. Les erreurs de prédiction de branchement qui en résultent en sont la cause probable ; nous n'avons pas pu lire les compteurs matériels sous macOS pour le confirmer.
2. **Le Bernoulli à chiffres précalculés est déjà au coût de la lecture de ses bits** (2,4 à 2,9 ns, contre 2,7 ns pour lire 18 bits du flux) : pas de boucle, un `ctz` et un décalage. La comparaison directe sur 31 bits (cmp31) est plus lente et consomme 15 fois plus de bits : ici, économiser les bits ne coûte pas de temps. Le Bernoulli recyclé prend le même temps pour 0,02 bit au lieu de 2. Sur `d.00150`, où les seuils sont moins extrêmes ($\mathbb{E}[h] = 0{,}18$), le résultat du Bernoulli devient moins prévisible et toutes les variantes ralentissent.
3. **Le recyclage avec deux états** est 1,8 fois plus rapide que la version actuelle sur `d.00100` (5,40 contre 9,68 ns) et consomme 3,3 bits de moins (14,34 contre 17,60). En temps réel, il est à 1 ns du plancher inexact ; en temps CPU, il est en dessous du plancher sauf sur `d.00150`, car il demande 4,5 fois moins de bits au producteur. Avec un seul état partagé, il est au contraire plus lent : le Bernoulli lit P_i en mémoire, et l'uniforme suivante attend l'état mis à jour par ce Bernoulli. La latence mémoire entre alors dans la chaîne de dépendances d'un tirage au suivant. Avec deux états, la chaîne de l'uniforme n'attend plus cette lecture, et le processeur recouvre les tirages successifs.

9.3 Remplacer les algorithmes de Lumbroso, au prix de bits

On peut aussi échanger des bits contre du temps, sans recyclage, en remplaçant chaque brique par un algorithme exact plus simple. Deux variantes de plus :

— **Lemire32** : uniforme de Lemire sur un mot de 32 bits (rejet de probabilité $< q/2^{32} \approx 5 \cdot 10^{-6}$), suivie du Bernoulli actuel.

— **Mot de 64 bits exact** : la version exacte du plancher. Les 32 bits de poids fort donnent la cellule (Lemire32), et les 32 bits de poids faible sont comparés aux 32 premiers chiffres de T_i/cs. En cas d'égalité (probabilité 2^{-32}), la suite est tirée exactement en GMP.

Temps (ns) / bits	`d.00000`	`d.00100`	`d.00150`	`d.00200`
FDR + Bernoulli `ctz` (actuel)	8,96 / 17,6	9,35 / 17,6	10,54 / 17,6	7,70 / 15,6
Lemire 64 + Bernoulli `ctz`	5,73 / 66,0	6,33 / 66,0	7,28 / 66,0	3,87 / 64,0
Lemire32 + Bernoulli `ctz`	5,54 / 34,0	5,98 / 34,0	6,80 / 34,0	3,34 / 32,0
Mot de 64 bits exact	**4,08** / 64,0	**4,39** / 64,0	**4,54** / 64,0	3,95 / 64,0
Recyclage, 2 états	4,80 / 14,3	5,11 / 14,3	6,90 / 14,5	4,88 / 14,3
Plancher inexact	3,91 / 64,0	4,34 / 64,0	4,23 / 64,0	4,47 / 64,0

TABLE 5 – Nouvelle série de mesures, mêmes conditions que la table 4. Temps CPU sur `d.00100` : actuel 10,0 ns, Lemire32 7,0 ns, mot de 64 bits 6,2 ns, recyclage 5,6 ns. χ^2 sur $3 \cdot 10^7$ tirages de `d.00100` et de `d.00150` : toutes les variantes exactes donnent des p-valeurs de 0,02 à 0,998 (24 tests), ce qui est compatible avec des lois exactes.

— **Uniforme.** C'est là que le remplacement rapporte. Seule, Lemire32 coûte 2,6 ns pour 32 bits, contre 6,8 ns pour 15,6 bits avec le FDR. Dans un tirage complet, on gagne 3,4 ns (9,35 → 5,98) pour 16 bits de plus.
— **Bernoulli.** Le remplacer seul ne rapporte rien : la comparaison sur 31 bits est plus lente que le Bernoulli à chiffres précalculés (3,6 contre 2,8 ns). Le Bernoulli n'est plus coûteux que s'il fait lire des bits en plus. C'est pourquoi la variante la plus rapide prend tout dans *un seul* mot de 64 bits : une seule lecture du flux, sans boucle, avec des branchements presque toujours prévisibles.
— **Mot de 64 bits exact.** C'est la variante la plus rapide en temps réel (2,1 à 2,3 fois plus rapide que l'actuelle sur les distributions non uniformes), au niveau du plancher inexact, et elle reste exacte. Elle consomme 64 bits par tirage, soit 3,6 fois plus. En temps CPU, elle est battue par le recyclage, parce que le producteur doit fournir ces bits supplémentaires (environ 0,8 ns par mot de 32 bits).

9.4 Conclusion

— **Remplacer les deux algorithmes de Lumbroso** par un tirage exact sur un mot de 64 bits (Lemire pour la cellule, comparaison sur 32 bits pour le seuil) divise le temps par plus de 2, contre 3,6 fois plus de bits. Le gain vient de l'uniforme ; le Bernoulli actuel est déjà rapide.
— **En bits**, l'uniforme FDR et le Bernoulli de Lumbroso sont optimaux au sens de Knuth–Yao pour un tirage isolé : aucun algorithme qui part de bits frais à chaque tirage ne fait mieux en moyenne. Ils ne le sont pas pour une suite de tirages : le recyclage atteint $\log_2 q + \mathbb{E}[h]$ (14,34 bits contre 17,60). L'écart restant avec $H(p)$ tient à la structure Alias elle-même (une cellule uniforme parmi $q \approx n$) ; seules les méthodes par arbre (FLDR, ALDR) descendent vers $H(p)$.
— **En temps**, notre implémentation n'est pas optimale. Le gain mesuré possible est de l'ordre de 1,8, et vient surtout de l'uniforme. Avec le recyclage, on arrive près du plancher que nous avons mesuré pour ce flux de bits. Au-delà, il reste la lecture des bits (environ 2,7 ns) et l'accès à une cellule (environ 1,5 ns), que toutes les méthodes partagent. Un générateur non cryptographique abaisserait ce plancher, mais changerait la source d'aléa de la comparaison.
— **Ce qu'il en coûte.** Le recyclage garde un état d'un tirage à l'autre (un par thread), ce qui sort du cadre « un tirage, des bits frais » des analyses de l'article : le nombre de bits par tirage n'est plus celui de la proposition 4, et la comparaison chemin par chemin avec l'original n'a plus de sens. Il ajoute 8 octets par cellule (le seuil P sur 64 bits) et un repli GMP de probabilité $\leq 2^{-31}$. Il s'applique aussi à Fraction-Alias, et en partie aux autres méthodes (uniforme de Rust-Alias).
— **Recommandation.** Garder `alias.integers_opt` pour les figures de l'article, puisque c'est l'algorithme analysé, et présenter le recyclage comme une perspective, avec ces chiffres. On peut aussi l'intégrer au pipeline comme échantillonneur supplémentaire.

10 Integer-Alias v2 : un mot de 64 bits par tirage

Suite à la section 9, nous avons ajouté un échantillonneur `alias.integers_v2` qui privilégie le temps, au prix de bits aléatoires. `alias.integers_opt` n'est pas modifié : il reste l'algorithme analysé dans l'article.

Algorithme. La table est celle d'`alias.integers_opt` (algorithme 1, même construction). Seuls l'uniforme et le Bernoulli de Lumbroso sont remplacés. Chaque essai lit *un seul* mot W de 64 bits du flux :

- **Cellule** : $i = \lfloor h\,q/2^{32} \rfloor$ avec $h = \lfloor W/2^{32} \rfloor$ (méthode de Lemire, une multiplication). Le rejet, de probabilité $< q/2^{32} \approx 5 \cdot 10^{-6}$, tire de nouveaux mots de 32 bits ; le seuil de rejet $(2^{32} - q) \bmod q$ est précalculé.
- **Bernoulli** : $\ell = W \bmod 2^{32}$ est comparé aux 32 premiers chiffres binaires $D_i = \lfloor T_i\, 2^{32}/cs \rfloor$, précalculés. Si $\ell < D_i$, on renvoie l'objet de la cellule ; si $\ell > D_i$, son alias. Si $\ell = D_i$ (probabilité 2^{-32}), on tire exactement Bernoulli$((T_i\, 2^{32} \bmod cs)/cs)$ en GMP avec des bits frais. C'est exact : avec $V = (\ell + \Theta)/2^{32}$ et Θ uniforme sur $[0, 1)$, le résultat vaut $\mathbf{1}\{V < T_i/cs\}$.
- Pas de boucle, et des branchements presque toujours prévisibles. Pour une cellule sans alias, l'alias vaut l'objet lui-même et $D_i = 2^{32} - 1$: la sélection se fait sans branchement.
- **Objet virtuel** : comme dans l'algorithme 1, si le résultat est l'objet virtuel, on recommence avec un nouveau mot (probabilité $\approx 5 \cdot 10^{-5}$ sur les distributions du protocole).

Le tirage complet (`c/alias_integers_v2_sample.c`), sans les deux replis rares :

```
do {
    uint64_t w = next_word();                // 64 bits du flux, d'un coup
    uint64_t m = (w >> 32) * q;              // Lemire sur les 32 bits de poids fort
    uint32_t i = m >> 32;
    if ((uint32_t)m < q) i = uniform_retry(m);          // rejet, proba < q / 2^32
    struct alias_v2_cell *c = &cells[i];
    uint32_t lo = (uint32_t)w;               // 32 bits de poids faible
    uint32_t b = lo < c->d32;                // Bernoulli(T_i / cs) sur 32 chiffres
    if (lo == c->d32) b = bernoulli_tie(i);  // egalite, proba 2^-32 : GMP exact
    result = b ? c->primary : c->alias;
} while (result == virtual_obj);
```

Pourquoi c'est plus rapide.

- **L'uniforme.** Le FDR lit un bit par itération après son premier bloc, et sa sortie de boucle dépend du hasard : c'était le poste le plus coûteux (environ 7 ns, section 9). Lemire le remplace par une multiplication, et son rejet n'arrive presque jamais.
- **Le Bernoulli.** Il ne lit plus de bits : il utilise les 32 bits de poids faible du mot déjà lu, et se réduit à une comparaison d'entiers.
- **La lecture des bits.** Le flux est lu par mots entiers de 64 bits, donc toujours aligné (`next_word`). La version actuelle lit des nombres de bits variables, qui franchissent régulièrement une frontière de mot.

Le prix est le nombre de bits : 64 par essai, sans réutiliser ceux dont l'uniforme et le Bernoulli n'ont pas besoin. Le producteur doit donc en fournir 3,6 fois plus, ce que `clock()` compte aussi (environ 1,6 ns par mot de 64 bits). La consommation est de 64 bits par essai (64,0 par tirage), contre 17,6 pour `alias.integers_opt`. La mémoire est la même à 24 octets près : cellules de 16 octets, avec D_i sur 32 bits à la place des 31 chiffres et du drapeau.

Exactitude.

- χ^2 sur $3 \cdot 10^7$ tirages de `d.00100`, `d.00150` et `d.00180` : $p = 0{,}15$, $0{,}97$ et $0{,}67$.
- Paires de tirages successifs (classes de masse $\approx 1/8$) : $p = 0{,}76$, $0{,}75$ et $0{,}65$.

— Cas d'égalité $\ell = D_i$: il est trop rare pour apparaître dans ces tirages, il a donc été testé à part. Sur 8 cellules de `d.00180`, 10^6 appels chacune, la fréquence de 1 coïncide avec $(T_i\, 2^{32} \bmod cs)/cs$ ($|z| < 0{,}6$).

Programmes : `bench_temps/check_v2.c` et `bench_temps/check_v2_tie.c`.

Mesures. Sur les 201 distributions du pipeline séquentiel (section 8), v2 prend 4,4 ns par tirage en médiane (de 4,1 à 7,1), contre 10,5 ns pour `alias.integers_opt`. Calculé distribution par distribution, le gain vaut 2,34 en médiane (de 1,46 à 3,91). Il ne dépend pas de l'entropie (figure 3b) : comme Integer-Alias, v2 tire une cellule uniforme quelle que soit la distribution. Les temps se répartissent sur deux paliers : 118 distributions entre 4,1 et 4,5 ns, les 83 autres dispersées jusqu'à 7,1 ns, à toutes les entropies. Ce n'est donc pas un effet de la distribution, mais de la machine. À ces temps, le thread producteur de bits, que `clock()` compte aussi, et la répartition des threads entre cœurs performance et efficacité pèsent sur la mesure.

Premières mesures, avec `main.out.opt`, 10^7 tirages, exécution seule, temps `clock()` :

ns par tirage	`d.00000`	`d.00100`	`d.00150`	`d.00200`
`alias.integers_opt`	9,63	10,48	11,40	8,26
`alias.integers_v2`	5,12	4,28	4,37	6,24

Une même distribution, mesurée trois fois, donne de 4,1 à 6,7 ns : c'est la même variabilité que les deux paliers ci-dessus.

Intégration.

— `c/alias_integers_v2.{h,c}` et `c/alias_integers_v2_sample.c` ; entrées dans `main.c`, `histogram.c`, `measure_sizes.c` et dans les deux Makefiles. `main.c` lui attribue le nouveau producteur de bits par son nom, puisqu'il ne finit pas par `_opt`.

— `pipeline.sh`, `dists.py` et `pipeline_seq.sh`. Ce dernier copie le fichier d'entrée depuis `.alias.integers_gmp`, qui a le même format, s'il manque.

— Notebook `figure-4-interpolation_Antoine.ipynb` : v2 apparaît en cyan. L'axe des temps descend à 2^{-29}. Ses 64 bits sont hors échelle sur la figure des bits (0 à 30) ; il apparaît sur `figure-4-interpolation-opt-entropy-bits-rust` (0 à 240 bits).

— **Correction du notebook.** Il associait temps, tailles et bits selon leur position dans des listes de fichiers triées. Si un fichier manquait pour un seul échantillonneur, les données des autres étaient décalées : nous l'avons constaté sur un dossier de test, où Rust-Alias apparaissait à 17 bits. Les tailles et les bits sont maintenant lus échantillonneur par échantillonneur, à partir du nom du fichier `.runtimes`.

11 Limites

— **Prétraitement.** Le précalcul des chiffres binaires coûte une division GMP par cellule. Mesuré seul, sans la lecture du fichier (section 8.1), le prétraitement d'Integer-Alias reste du même ordre que l'original : de 0,1 à 1,6 ms, contre environ 1 ms. Une estimation antérieure de ce rapport (3,9 puis 13 ms) ne séparait pas la construction du reste de l'exécution, et était trop élevée. Les versions optimisées de Rust-Alias et Fraction-Alias réutilisent la construction d'origine puis convertissent la table : leur prétraitement est un peu plus long que l'original. À l'inverse, celui d'ALDR est plus rapide (une multiplication par objet au lieu d'une par objet et par niveau).

— **Comparaison du producteur.** Les versions optimisées utilisent le nouveau producteur, les originaux l'ancien. Le gain total mélange donc les deux effets ; la décomposition donnée en section 1 pour Integer-Alias en donne l'ordre de grandeur.

— **Tailles.** Les indices sont sur 32 bits : tables limitées à $2^{32} - 1$ cellules ; un message d'erreur explicite est émis au-delà.

12 Fichiers et reproduction

Nouveaux fichiers (`c/`). `flip_opt.{h,c}` (producteur et briques communes), `alias_integers_opt.{h,c}`, `alias_integers_opt_sample.c`, `fldr_opt.{h,c}`, `aldr_opt.{h,c}`, `alias_rust_opt.{h,c}`, `alias_fractions_opt.{h,c}`, `alias_integers_v2.{h,c}`, `alias_integers_v2_sample.c`, et les fichiers de test `d/d.00050.*_opt`.

Section 9. `rapport_optimisation/bench_temps/` : `bench.c` (variantes et mesures), `theory.py` (bits exacts du FDR et de l'arbre KY, $\mathbb{E}[h]$ d'une table), `chi2.py` (tests), `res/` (sorties brutes). Compilation, depuis ce dossier :

```
gcc -O3 -I../../c -o bench.out bench.c ../../c/alias_integers_opt.c \
    ../../c/alias_integers_opt_sample.c ../../c/flip_opt.c \
    $(pkg-config --cflags --libs gmp gsl)
./bench.out ../../experiments/distributions_entropy/d.00100.alias.integers_opt 20000000
```

Preprocessing. `c/measure_preprocess.c` (cible `measure_preprocess.out.opt` des Makefiles) ; fonctions `preprocess_fldr_opt`, `preprocess_alias_integers_v2`, `preprocess_alias_rust_opt` et `preprocess_alias_fractions_opt` ajoutées aux fichiers `_opt` et v2.

Pipeline séquentiel. `experiments/pipeline_seq.sh` produit les mêmes fichiers que `pipeline.sh`, pour les cinq versions `_opt` et `alias.integers_v2` seulement ; il lance une seule mesure à la fois et mesure chaque échantillonneur une fois, en alternant les échantillonneurs pour chaque distribution.

Modifications du code existant.

- `c/sample.c` : correction du parcours de `sample_ky_encoding_gmp` (trois lignes, section 5). C'est la seule modification d'un algorithme d'origine.
- `c/main.c`, `c/histogram.c` : entrées des cinq échantillonneurs `_opt`, et choix du nouveau producteur pour eux ; `c/measure_sizes.c` : leurs tailles.
- `c/Makefile` et `c/Makefile_Tan` : nouvelles sources.
- `experiments/pipeline.sh` et `experiments/dists.py` : les cinq échantillonneurs, avec les mêmes fichiers d'entrée que leurs versions `_gmp`.
- `experiments/notebooks/figure-4-interpolation_Antoine.ipynb` : labels, sélection, une teinte par algorithme, axes élargis et légendes sur deux colonnes.

Reproduction. Pour relancer toutes les mesures :

```
cd c && make -f Makefile_Tan main.out.opt measure_sizes.out.opt histogram.out
cd ../experiments
./pipeline.sh distributions_entropy measure-runtimes 10000000
./pipeline.sh distributions_entropy measure-structure-sizes
./pipeline.sh distributions_entropy aggregate-runtimes 10000000
```

ou, pour les seules versions optimisées et v2, sans mesures en parallèle (environ 9 minutes) :

```
caffeinate -i ./pipeline_seq.sh distributions_entropy run-all 10000000
```